

Day 60: Gradient Clipping

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I (Day 5 of 7)

0. Where Today Fits

Day 59 left two problems at T=20: gradient reaching early timesteps was small at initialization, and the global gradient norm spiked up to 47x its median as W_{hh} 's spectral radius crossed about 2. Gradient clipping is the standard fix for the second. Day 59 predicted clipping would tame the spikes but NOT rescue learning, since clipping can only shrink gradients. That prediction was wrong -- and working out why corrects Day 59 itself.

How to read each section

Every experiment runs **Specification** → **Why this test** → **Implementation** (the real code from `day60_gradient_clipping.py`) → **Actual output** (verbatim from `day60_run_output.txt`) → **What it means**. Try to predict each output before reading it.

Watch: [Gradient Clipping for Neural Networks | Deep Learning Fundamentals](#)

Watch: [Stanford CS224N: NLP with Deep Learning | Winter 2019 | Lecture 7 – Vanishing Gradients, Fancy RNNs](#) (covers clipping for RNNs)

1. Clipping by Global Norm -- Built and Checked

MATH

Treat every parameter's gradient as one long vector g . Its global norm is $\|g\| = \sqrt{\text{sum of squares of every entry}}$. Given a threshold c :

- if $\|g\| \leq c$: use g unchanged
- if $\|g\| > c$: use $g * (c / \|g\|)$

Every entry is multiplied by the SAME factor, so the direction is unchanged -- only the step length is capped, at $\text{lr} * c$. Element-wise VALUE clipping (clip each entry to $[-\text{limit}, \text{limit}]$ independently) cuts large entries but leaves small ones alone, which changes the direction.

SPECIFICATION

Gradient tested	one real gradient from Day 59's network (seed 7) on 30 sign-task sequences of length 20 -- all five parameters, 322 entries
Thresholds	$c = 10$ and $2x$ the norm (both above it), $0.5x$ the norm and 0.1 (both below it)
Checked	norm after clipping; the scale factor; cosine similarity with the original (1 = same direction); whether the exact same arrays came back (a true no-op)
Contrast	value clipping at limits 0.05, 0.01, 0.002: norm after, cosine, how many entries changed

Why this test

Clipping introduces no new derivative, so there's no gradient to check. What must hold instead are its three defining properties -- and a comparison showing why the global-norm version is the one to use.

IMPLEMENTATION

```
def clip_by_global_norm(grads, max_norm):
    norm = np.sqrt(sum(np.sum(g ** 2) for g in grads.values())) # all gradients as one vector
    if max_norm is None or norm <= max_norm:
        return grads, norm, 1.0 # the ORIGINAL arrays -- an exact no-op
    scale = max_norm / norm # makes the norm exactly max_norm
    return {k: g * scale for k, g in grads.items()}, norm, scale # one factor for every entry

def clip_by_value(grads, limit): # the alternative, for contrast
    return {k: np.clip(g, -limit, limit) for k, g in grads.items()}

# inside ClippedRNClassifier.backward(), between computing and applying gradients:
grads, self.pre_clip_norm, self.scale = clip_by_global_norm(grads, self.clip)
for name, g in grads.items():
    getattr(self, name)[...] -= self.lr * g
```

ACTUAL OUTPUT

raw gradient: 322 entries, global norm 1.2143

threshold c	norm after	scale	cosine	unchanged?
10.0000	1.2143	1.0000	1.00000000	True
2.4287	1.2143	1.0000	1.00000000	True
0.6072	0.6072	0.5000	1.00000000	False
0.1000	0.1000	0.0823	1.00000000	False

ACTUAL OUTPUT

contrast -- element-wise value clipping to [-limit, limit]:

limit	norm after	cosine	entries changed
0.05	0.6865	0.895447	134 of 322
0.01	0.1714	0.788184	273 of 322
0.002	0.0358	0.759369	318 of 322

What it means

Above the norm, clipping returns the identical arrays (True). Below it, the norm lands exactly on c and the cosine is 1.00000000 -- same direction, shorter step. Value clipping bends the direction (cosine 0.76-0.90), because it changes the RATIOS between entries: large ones are cut, small ones aren't.

2. Before/After on Day 59's Exact Run

SPECIFICATION

Run	Day 59 Section 5 exactly: sign task, T=20, train 600 (seed 42), test 400 (seed 999), hidden 16, default init (seed 7), lr 0.05, 150 epochs, batch 30, same shuffling
Before	clip = None -- must reproduce Day 59's final test accuracy of 0.4950 exactly
After	clip = 1.0 -- the ONLY difference
Recorded every step	pre-clip gradient norm, length of the step actually taken (lr x norm x scale), whether it was clipped
Recorded every 5 epochs	test accuracy, Whh's spectral radius, and the gradient ratio $\ dL/dh_0\ / \ dL/dh_{19}\ $ -- Day 59 only measured these at the end

Why this test

The consistency check proves the comparison is fair: if the unclipped run doesn't reproduce Day 59 exactly, something other than clipping differs. Tracking accuracy DURING training -- not just at the end -- is the measurement Day 59 didn't take, and it turns out to be the one that matters.

IMPLEMENTATION

```
for label, c in [("unclipped", None), ("clip=1.0", 1.0)]:
    net = ClippedRNNCClassifier(lr=0.05, clip=c, seed=7)      # identical except the threshold
    steps, ev = train_tracked(net, X_tr, y_tr, X_te, y_te)    # evaluates every 5 epochs

# locating the collapse in the unclipped run
peak_i = int(np.argmax(eu["test"]))
crash = next(i for i in range(peak_i, len(eu["test"]))) if eu["test"][i] < 0.6
lo, hi = eu["step"][crash - 1], eu["step"][crash]           # the window it happened in
biggest = lo + int(np.argmax(su["pre"][lo:hi]))             # largest gradient in that window
```

ACTUAL OUTPUT

consistency check -- unclipped final test accuracy 0.4950 (Day 59 reported 0.4950 for this exact run)

ACTUAL OUTPUT

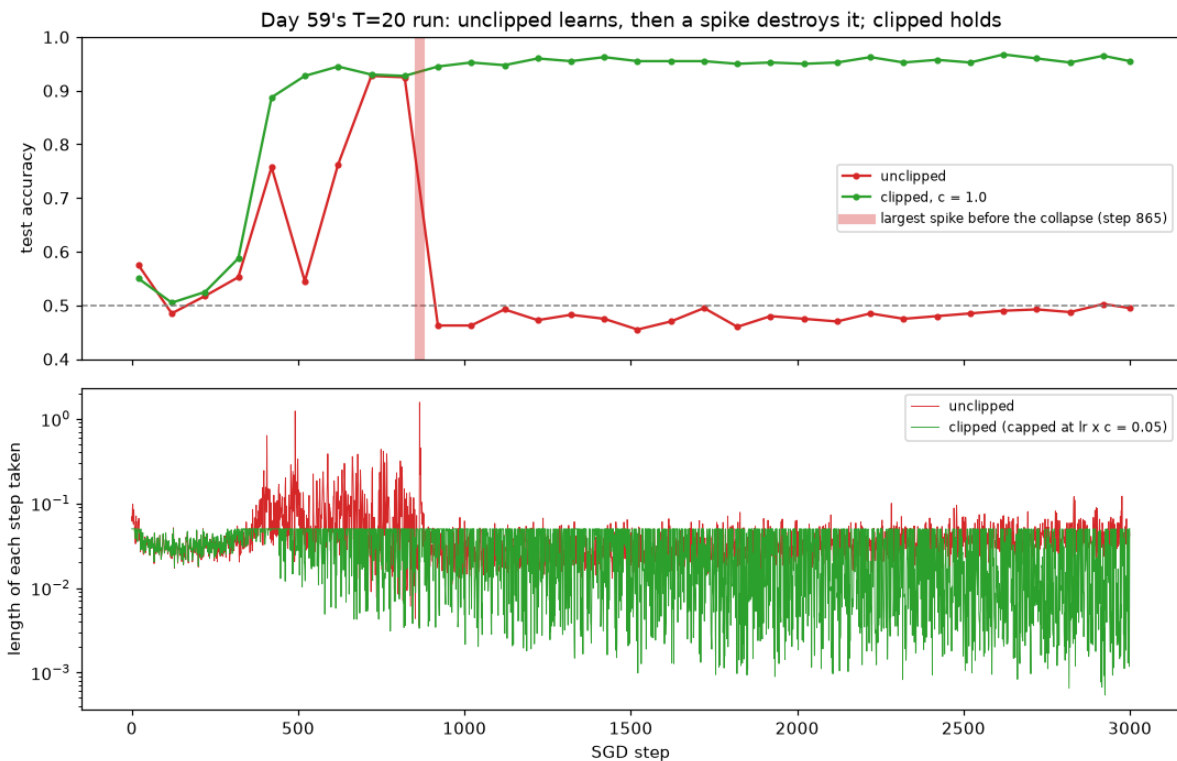
step	unclipped: test	rho	ratio	clip=1.0: test	rho	ratio
20	0.575	1.27	3.42e-02	0.550	1.28	3.58e-02
220	0.517	1.35	6.50e-02	0.525	1.37	1.05e-01
420	0.757	1.95	2.86e+00	0.887	1.95	9.16e+00
620	0.762	2.48	3.22e+00	0.945	2.15	3.88e+01
820	0.925	2.92	1.09e+01	0.927	2.23	1.76e+01
1020	0.463	2.83	2.80e-04	0.953	2.27	2.24e+01
1220	0.472	2.83	3.13e-04	0.960	2.30	2.51e+01
1420	0.475	2.84	3.78e-04	0.963	2.33	2.00e+01
1620	0.470	2.84	4.60e-04	0.955	2.35	2.74e+01
1820	0.460	2.85	6.45e-04	0.950	2.38	4.45e+01
3000	0.495	2.85	7.80e-03	0.955	2.49	3.65e+01

ACTUAL OUTPUT

	unclipped	clip=1.0
peak test accuracy	0.927	0.968
final test accuracy	0.495	0.955
final train accuracy	0.678	0.993
final spectral radius	2.85	2.49
largest step taken	1.572	0.050
steps clipped	0.0%	33.5%

ACTUAL OUTPUT

the unclipped collapse: test accuracy 0.925 at step 820 -> 0.463 at step 920; the largest gradient in that window was 31.4 (47x the run's median), and it was applied as a step of length 1.572



What it means

The unclipped network LEARNED: 0.925 test accuracy at step 820, with the gradient ratio reaching step 0 at $1.09e+01$ -- gradient was flowing all the way back. Then one gradient of 31.4 (47x the median) was applied at step 865 as a step of length 1.572, and by step 920 accuracy was 0.463. The ratio collapsed to $2.80e-04$: the step threw the network into a saturated state where gradient vanishes again, and it never recovered. Clipped, no step is longer than 0.05; the network crosses the same region intact and finishes at 0.955 (peak 0.968), clipping about a third of its steps.

WHY THIS CHOICE

This corrects Day 59. Day 59 measured the T=20 network only at the end, found it had never stored `x_0`, and blamed vanishing gradients. That describes the damaged network, not the one that was learning. The real sequence: vanishing gradients slowed the start; training grew Whh's radius until gradient reached step 0; the network learned; an exploding-gradient spike destroyed it. Day 59's guide now carries an update box saying so. The general lesson: an end-of-training snapshot can completely misattribute a failure.

3. How Robust Is the Rescue?

SPECIFICATION

Run	Section 2's run, unchanged except the threshold
Thresholds	None, 0.5, 1.0, 2.0, 5.0, 10.0
Measured	peak test accuracy at any evaluation, final test accuracy, final spectral radius

Why this test

A result at one hand-picked threshold could be luck. If the rescue holds across a 20x range of thresholds, it is a property of clipping, not of the number chosen.

IMPLEMENTATION

```
def summarize(T, lr, clip):
    net = ClippedRNNClassifier(lr=lr, clip=clip, seed=7)
    _, ev = train_tracked(net, X_tr, y_tr, X_te, y_te)
    return ev["test"].max(), ev["test"][-1], ev["rho"][-1]    # peak, final, final radius

for c in [None, 0.5, 1.0, 2.0, 5.0, 10.0]:
    print(c, *summarize(20, 0.05, c))
```

ACTUAL OUTPUT

c	peak test	final test	final rho
None	0.927	0.495	2.85
0.5	0.958	0.953	2.36
1.0	0.968	0.955	2.49
2.0	0.970	0.970	2.64
5.0	0.978	0.968	3.06
10.0	0.983	0.975	3.42

What it means

Every threshold works: finals from 0.953 to 0.975, all within a few points of their peaks. Only the unclipped run shows the peak-then-collapse pattern (0.927 -> 0.495). Larger thresholds allow a larger final radius (2.36 at `c=0.5` up to 3.42 at `c=10`) -- they let training move further -- without the destructive single steps.

4. Clipping Makes Larger Learning Rates Usable

SPECIFICATION

Length	T = 10 -- where Day 59's lr=0.05 run learned fine (test 0.990)
Learning rates	0.2, 0.5, 1.0 (4x, 10x, 20x Day 59's)
Compared	unclipped vs. clip = 1.0; final test accuracy and final spectral radius

Why this test

A larger learning rate multiplies every step, including the explosive ones. This asks whether clipping lets a network use bigger steps safely on a task it can otherwise learn.

ACTUAL OUTPUT

lr	unclipped final	clipped final	unclipped rho	clipped rho
0.2	0.688	0.990	11.61	2.93
0.5	0.487	0.995	9.60	5.16
1.0	0.545	0.963	1406.76	10.24

What it means

Unclipped, every larger learning rate fails (0.488 to 0.688), and the recurrent weights blow up -- at lr=1.0 Whh's spectral radius reaches 1,406.76. Clipped, all three learn (0.963 to 0.995) with radii of 2.93 to 10.24. Capping the step length is what makes the larger learning rate survivable.

5. The Limit -- T=40

SPECIFICATION

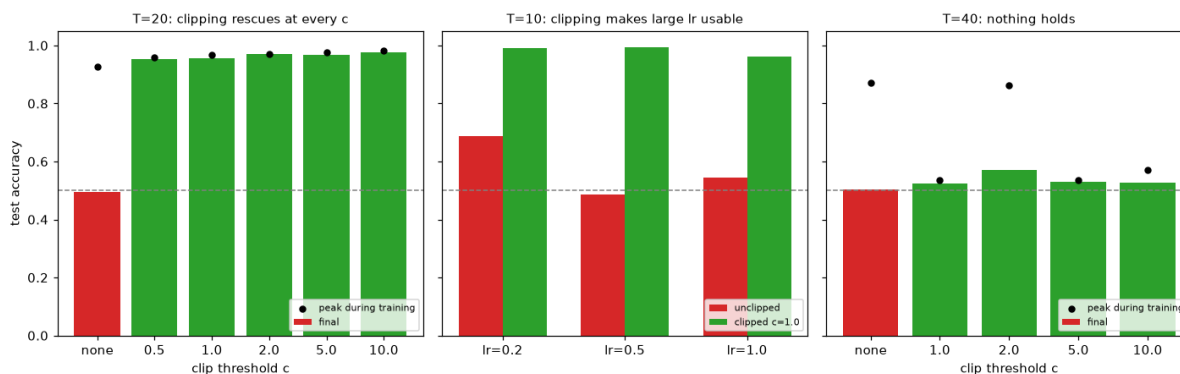
Length	T = 40 -- Day 59's other failing length (gradient ratio at initialization 3.3e-4)
Thresholds	None, 1.0, 2.0, 5.0, 10.0; everything else as in Section 3
Success means	a final test accuracy well above 0.5 -- learning that HOLDS, not just a peak

Why this test

Section 2 showed clipping fixes a collapse caused by a spike. Clipping can't enlarge a gradient that is too small to start learning. T=40 tests where that limit bites.

ACTUAL OUTPUT

c	peak test	final test	final rho
None	0.870	0.502	2.55
1.0	0.535	0.525	1.77
2.0	0.863	0.570	2.17
5.0	0.535	0.530	2.00
10.0	0.570	0.527	1.98



What it means

No threshold holds. Unclipped, the network peaks at 0.870 and collapses to 0.502 -- the same pattern as $T=20$. With clipping it either never really learns ($c=1.0, 5.0, 10.0$ peak at 0.535-0.570) or learns and loses it anyway ($c=2.0$: 0.863 $\rightarrow$ 0.570). Results jump around between thresholds -- the sign of a fragile regime, not a tunable one. At this length the vanilla RNN's own recurrence is the bottleneck, and no optimizer tweak fixes that.

6. What Comes Next

Clipping caps the length of a step; it cannot make a vanishing gradient larger. At $T=20$ that was enough, because the danger was the crossing itself. At $T=40$ it wasn't. The fix there is a change to the recurrence -- the LSTM's gated cell state (Day 63). First, Days 61-62 put the vanilla RNN to work on real many-to-one and many-to-many tasks.

Interview Preparation -- Technical Questions

Q: How does gradient clipping by global norm work?

A: Compute the norm of all gradients as one vector. If it exceeds a threshold c , multiply every gradient by c / norm . The step's length becomes exactly $lr * c$ and its direction is unchanged.

Q: Why clip by global norm rather than clipping each value?

A: Value clipping cuts large entries but not small ones, changing the ratios between them -- the direction changes (cosine 0.76-0.90 in this lesson). Global-norm clipping scales every entry equally, so the direction is exactly preserved (cosine 1.00000000).

Q: Clipping can only shrink gradients. How did it rescue a run Day 59 blamed on vanishing gradients?

A: Because the run hadn't failed from vanishing gradients. Tracking accuracy during training showed it learned to 0.925, then a single 31.4 gradient applied as a 1.572-length step knocked it to 0.463. Clipping capped that step at 0.05, so the collapse never happened.

Q: Why check that the unclipped run reproduces Day 59 exactly?

A: It proves the before/after differ ONLY in clipping. Any mismatch would mean some other difference could explain the result.

Interview Preparation -- Theory Questions

Q: What problem was gradient clipping introduced to solve?

A: Exploding gradients -- in particular the sharp 'cliffs' in an RNN's loss surface described by Pascanu, Mikolov and Bengio (2013), where a single step can throw the parameters far from a good region.

Q: What can't gradient clipping fix?

A: Gradients that are too SMALL. At T=40 no threshold produced a network that learned and stayed learned. Long-range dependencies that vanilla RNNs can't carry need an architectural fix such as the LSTM.

Q: What does this lesson say about diagnosing training failures?

A: Look at the whole trajectory. Day 59's end-of-training measurements pointed to vanishing gradients; the trajectory showed learning followed by a collapse -- a completely different cause with a different fix.

Key Takeaway and What's Next

Key takeaway: clipping by global norm caps each step's length while keeping its direction exactly. On Day 59's run it turned a network that learned and then collapsed (0.925 -> 0.495) into one that holds (0.955), at every threshold from 0.5 to 10; it also made 4-20x larger learning rates usable. It could not rescue T=40, where the vanilla RNN itself is the limit. And it corrected Day 59: that failure was an exploding-gradient collapse, visible only by watching training rather than its end state.

What's next: Day 61 trains a many-to-one RNN classifier on a real short-text sentiment dataset.

Suggested watch order: 1) Gradient Clipping for Neural Networks, 2) Stanford CS224N Lecture 7 (clipping section).